1. Introduction to Events

- **Importance**: Events are a crucial topic in JavaScript. Mastering them enables building projects without issues. The speaker calls it a "lallu sa topic" (simple topic) but stresses diving deep for conceptual clarity.
- **Goal of the Class**: Learn events thoroughly to start projects in the next class.
- **Basic Definition**: Events are user interactions with the browser, such as mouse movements, clicks, double-clicks, scrolls, or keyboard inputs. "इवेंट आपके कुछ नहीं है इवेंट है आपके सिंपल से कि अगर आप माउस को मूव कर रहे हैं या फिर क्लिक कर रहे हैं या डबल क्लिक कर रहे हैं ये सब कुछ आपके इवेंट का पार्ट होते हैं।"
- **Three Key Phases of Event Handling** (High-Level):
    1. **Event Occurrence**: The action happens (e.g., click).
    2. **Event Listening**: Attach a listener to detect the event on a specific element.
    3. **Event Action**: Perform a response (e.g., change text).
- **Example Setup**: A simple HTML page with an <h1 id="first">Hello Coder Army</h1>, styled with black background and white text. On click, change text to "Strike is Coming".

2. Methods to Handle Events

The transcript compares three ways to attach event handlers, explaining why addEventListener is best.

2.1 Inline Event Handler (Not Recommended)

- Example: <h1 id="first" onclick="handleClick()">Hello Coder Army</h1>
- In JS: Define function handleClick() { document.getElementById('first').textContent = 'Strike is Coming'; }
- Issue: Mixes JavaScript in HTML, violating separation of concerns. "ये जावास्क्रिप्ट का कोड मैं यहां पे नहीं लिखना चाहता। जावास्क्रिप्ट का कोड यहीं पे लिखा जाए भाई।"

2.2 Using element.onclick (Also Not Recommended)

Example:
JavaScript

```
const element = document.getElementById('first');
```

- element.onclick = function() { element.textContent = 'Strike is Coming'; };
- Issue: Overwrites previous handlers if multiple are added. It's a property assignment, so later assignments override earlier ones. "नीचे वाला पहले वाले को ओवराइट कर देगा।"
- Demonstration: Adding two onclick handlers results in only the last one executing (e.g., text change overrides background color change).

2.3 Using addEventListener (Recommended)

- Syntax: element.addEventListener('eventType', callbackFunction, useCapture); (useCapture is optional, defaults to false).

Example:
JavaScript

```
const element = document.getElementById('first');
element.addEventListener('click', function() { element.textContent = 'Strike is Coming'; });
```

- element.addEventListener('click', function() { element.style.backgroundColor = 'brown'; });
- Advantages:
  - Allows multiple listeners on the same event without overwriting (calls all callbacks).
  - "यह सबसे बेस्ट है।" Because it's a method call, not property assignment. Multiple calls stack up.
  - Analogy: Like calling a method multiple times on an object (e.g., a.greet(10); a.greet(20); prints both).
- Comparison Table:

Method	Allows Multiple Handlers?	Separation of Concerns?	Overwrite Risk?	Recommendation
Inline (onclick attribute)	No (single function call)	Poor (JS in HTML)	High	Avoid
element.onclick	No (property override)	Better	High	Avoid
addEventListener	Yes (stacks callbacks)	Best	None	Use Always

3. Event Types

- Common Events: Click, dblclick (double-click), mouseenter (hover in), mouseleave (hover out), mousemove, scroll, keydown, etc.
- Examples from Transcript:
  - Change on 'dblclick': addEventListener('dblclick', ...) – Text changes only on double-click, not single.
  - Change on 'mouseenter': Text changes when mouse enters the element.
  - Change on 'mouseleave': Text changes when mouse leaves.
- Advice: No need to memorize; look up on the internet (e.g., MDN). "रटने की जरूरत नहीं है आपको। ये आप इंटरनेट से देख सकते हैं।"

4. Event Delegation

- **Concept**: Instead of attaching listeners to each child element, attach one to the parent and use event.target to identify the clicked child.
- Example Setup: A parent with five child s (ids: child1 to child5, classes: child, colored differently).

Inefficient Way: Loop through parent.children and add listener to each child (results in 5 listeners).
JavaScript

```
for (let child of parent.children) {
  child.addEventListener('click', function() { child.textContent = 'I am Clicked'; });
```

- }

Efficient Way (Delegation): Attach to parent, use e.target.
JavaScript

```
const parent = document.getElementById('parent');
```

- parent.addEventListener('click', function(e) { e.target.textContent = 'I am Clicked'; });
- Benefits:
    - Optimizes performance (1 listener vs. many).
    - Handles dynamic children (added later).
    - "फिर भी हमारे पांच इवेंट लिसनर लग रहे हैं। ... इससे भी बढ़िया ऑप्टिमाइज अप्रोच हमारे पास है। क्या है भैया वो? अब हम सीखने वाले हैं किसके बारे में? बबलिंग के बारे में।"
- Ties into bubbling (events bubble up to parent).

5. The Event Object

- Passed automatically to the callback: function(e) { ... } (e is the event object).
- Properties:
    - e.type: Event type (e.g., 'click').
    - e.clientX, e.clientY: Mouse coordinates relative to viewport.
    - e.target: The exact element that triggered the event (key for delegation).
    - Many more (e.g., pointer details).
- Example: console.log(e); shows a PointerEvent object with all info.
- Use: "यह आपको एक इवेंट का ऑब्जेक्ट भी ला के देता है जिसके अंदर बहुत सारी इंफॉर्मेशन प्रेजेंट रहती है।"

6. Stopping Propagation

- Method: e.stopPropagation();
- Stops bubbling (prevents event from reaching ancestors).
- Example: In child's listener: function(e) { ...; e.stopPropagation(); } – Parent/grandparent listeners won't fire.
- "मैं इस प्रोपोगेशन को रोक भी सकता हूं। ... ई डॉट स्टॉप प्रोपोगेशन"

7. Removing Event Listeners

- Syntax: element.removeEventListener('eventType', callbackFunction);
- Key: Must use the same function reference (not inline anonymous functions, as they have different memory).

Correct Way: Define named function.
JavaScript
```
function handleClick(e) { e.target.textContent = 'I am Clicked'; parent.removeEventListener('click', handleClick); }
```

- `parent.addEventListener('click', handleClick);`
- This allows one-time execution (e.g., remove after first click).
- Incorrect: Anonymous functions won't match for removal.

8. Deep Dive: Capturing Phase, Target Phase, and Bubbling Phase (Very Detailed)

This is the core focus as requested. The transcript explains these phases in depth, using a nested div example (grandparent > parent > child) with listeners on all. It emphasizes logical understanding ("कॉमन सेंस") over memorization, including behind-the-scenes DOM traversal.

8.1 Setup for Explanation

- HTML: Nested divs –   (styled as boxes: orange 300px > red 200px > green 100px).

JS: Add listeners to all:
JavaScript
```
grandparent.addEventListener('click', function(e) { console.log('Grandparent is Clicked'); }, false);  // Default false
parent.addEventListener('click', function(e) { console.log('Parent is Clicked'); }, false);
```

- `child.addEventListener('click', function(e) { console.log('Child is Clicked'); }, false);`
- Behavior: Clicking child logs: "Child is Clicked" > "Parent is Clicked" > "Grandparent is Clicked" (bubbling order).

8.2 The Three Phases in Depth

Events propagate through the DOM tree in three phases. The browser knows the target element immediately but traverses the tree logically.

Phase 1: Capturing Phase (Top-Down Traversal, "Capture" or "Trickle Down")

- **Description**: Starts from the root (window) and goes down to the target element, checking for listeners with useCapture: true.
- **Process**:
    - Browser detects click and identifies target (e.g., child).
    - Traverses from root downward: window > document > html > body > grandparent > parent > child (target).
    - At each ancestor, if a listener is attached with useCapture: true (third arg true), execute it immediately.
- **Default**: Off (false), so rarely executes here unless specified.
- **Transcript Quote**: "सबसे पहले वो स्टार्ट करेगा कैप्चर फेस को। ... विंडो से आप आए डॉक्यूमेंट पे, डॉक्यूमेंट से गए आप एचटीएमएल पे, एचटीएमएल से गए आप बॉडी पे, बॉडी से आए आप ग्रैंड पेरेंट पे। ग्रैंड पेरेंट्स आए पैरेंट पे। और फिर फाइनली आप आए किस पे? चाइल्ड पे जो कि आपका टारगेट है।"
- **Why?**: Allows ancestors to "capture" the event before it reaches the target. Useful for intercepting events early (rare in practice).
- **Effect of useCapture**:
    - true: Listener executes during capturing (top-down order).
    - Example: Set all to true – Clicking child logs: "Grandparent" > "Parent" > "Child" (capturing order).
- **Deep Insight**: If useCapture is true on an ancestor, it fires as the event "descends" the tree. No execution if false (saved for bubbling). "अगर आपका कैप्चर फेस ऑन है तो जब आप टॉप से नीचे आओगे, डाउन आओगे। उस टाइम पर आपके इवेंट को ट्रिगर कर दिया जाएगा।"

Phase 2: Target Phase (At the Exact Element)

- **Description**: Occurs when the traversal reaches the target element (e.g., child).
- **Process**:
    - Execute all listeners on the target itself, regardless of useCapture (true or false).
    - Order: Listeners with useCapture true execute first (if any), then false.
- **Transcript Quote**: "जैसे ही उसको आपका चाइल्ड मिल गया। अब यहां से स्टार्ट होता है आपका बबल फेस। ... सबसे पहले दिखेगा क्या चाइल्ड के ऊपर आपने कोई इवेंटेशन लगाया है उसने कहा हां लगाया है तो उसको एग्जीक्यूट कर दो।"
- **Deep Insight**: This is the "at-target" stop. The event is now at its origin. If multiple listeners on target, they all fire here. Transition point from capturing (down) to bubbling (up).

Phase 3: Bubbling Phase (Bottom-Up Propagation, "Bubble Up")

- **Description**: Starts from the target and bubbles up to the root, executing listeners with useCapture: false (default).
- **Process**:
    - From target upward: child > parent > grandparent > body > html > document > window.
    - At each ancestor, if listener with useCapture false, execute it.
- **Transcript Quote**: "अब यहां से बबलिंग होगा। बबलिंग क्या करता है? अंदर से बाहर की ओर जाता है। ... पहले मुझे ये दिखा आउटपुट में फिर ये दिखा फिर ये दिखा।"

- **Why Bubbling Happens**: Logical – Child is part of parent, so event on child affects parent too (common sense). "क्योंकि वो अगर आपके अंदर वो इवेंट ट्रिगर हुआ है तो वो किसके अंदर रखा होगा? वो पैरेंट के अंदर रखा होगा।"
- **Default Behavior**: Most listeners (false) execute here, in bottom-up order.
- **Stopping It**: e.stopPropagation() halts bubbling (event doesn't reach higher ancestors).
- **Deep Insight**: Bubbling enables delegation. Without listeners on ancestors, nothing happens during bubble (e.g., no log on body if no listener). "अगर कैप्चर फेस ऑफ है तो आपका जो भी इवेंट है उसको डाउन टू अप में जिसको हम बबलिंग फेस भी कहते हैं। तब ट्रिगर करा जाएगा।"
- **Mixed useCapture Example** (From Transcript):
    - Grandparent: true (captures during down phase).
    - Parent: false (bubbles during up phase).
    - Child: true (captures at target).
    - Click child: Logs "Grandparent" (capture down) > "Child" (target) > "Parent" (bubble up).
    - Reasoning: During capture: Hits grandparent (true, execute), skips parent (false), hits child (true, execute). Then bubble: Hits parent (false, execute now).

8.3 Key Takeaways on Phases

- **Overall Flow**: Capture (down, if true) > Target (all) > Bubble (up, if false).
- **useCapture Parameter**: Toggles phase execution. Default false (bubble). True = capture. "इसका मतलब है कि क्या आपका कैप्चर फेस ऑन है या ऑफ है?"
- **Performance/Use**: Bubbling common; capturing rare. "वैसे आप कैप्चर फेस को ज्यादा इस्तेमाल करते नहीं है। आप बाय डिफॉल्ट फॉल्स को ही प्रयोग करते हैं।"
- **Common Sense Analogy**: Events always happen (even without listeners). Listening decides response. Propagation is natural (child affects parent).
- **Events Without Listeners**: Still occur, but ignored. "इवेंट तो होते रहेंगे ना दोस्त। ... कोई सुन नहीं रहा तो इसमें मेरी गलती थोड़ी है।"